

TDVI: Inteligencia Artificial

Funciones de Activación

Licenciatura en Tecnología Digital

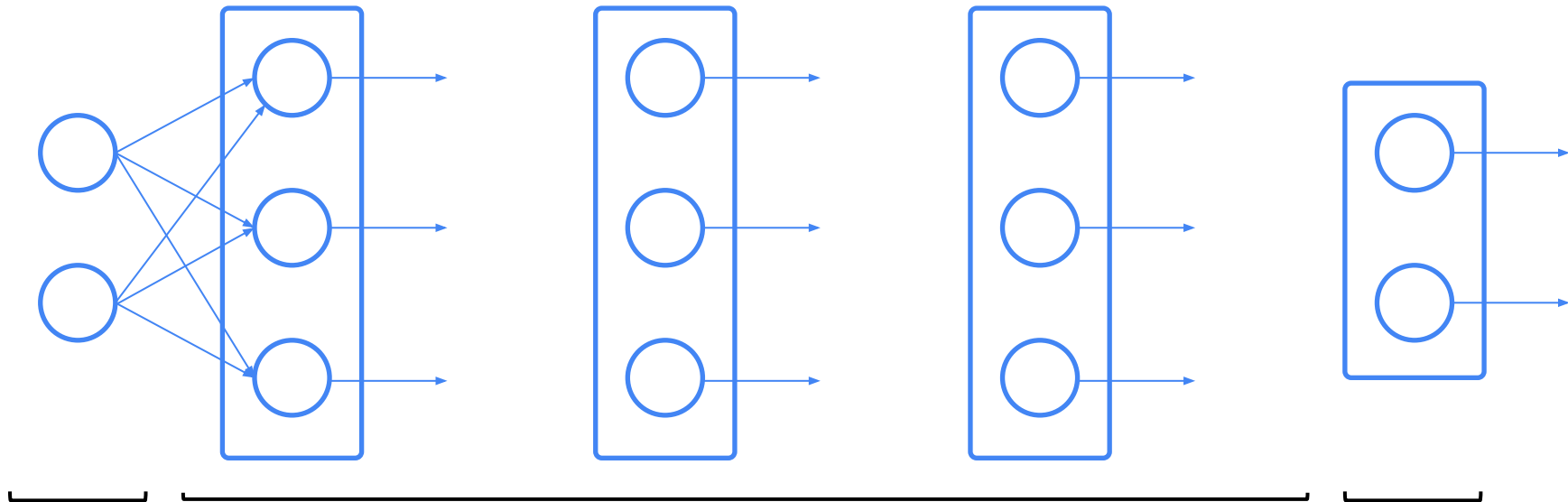


Agenda

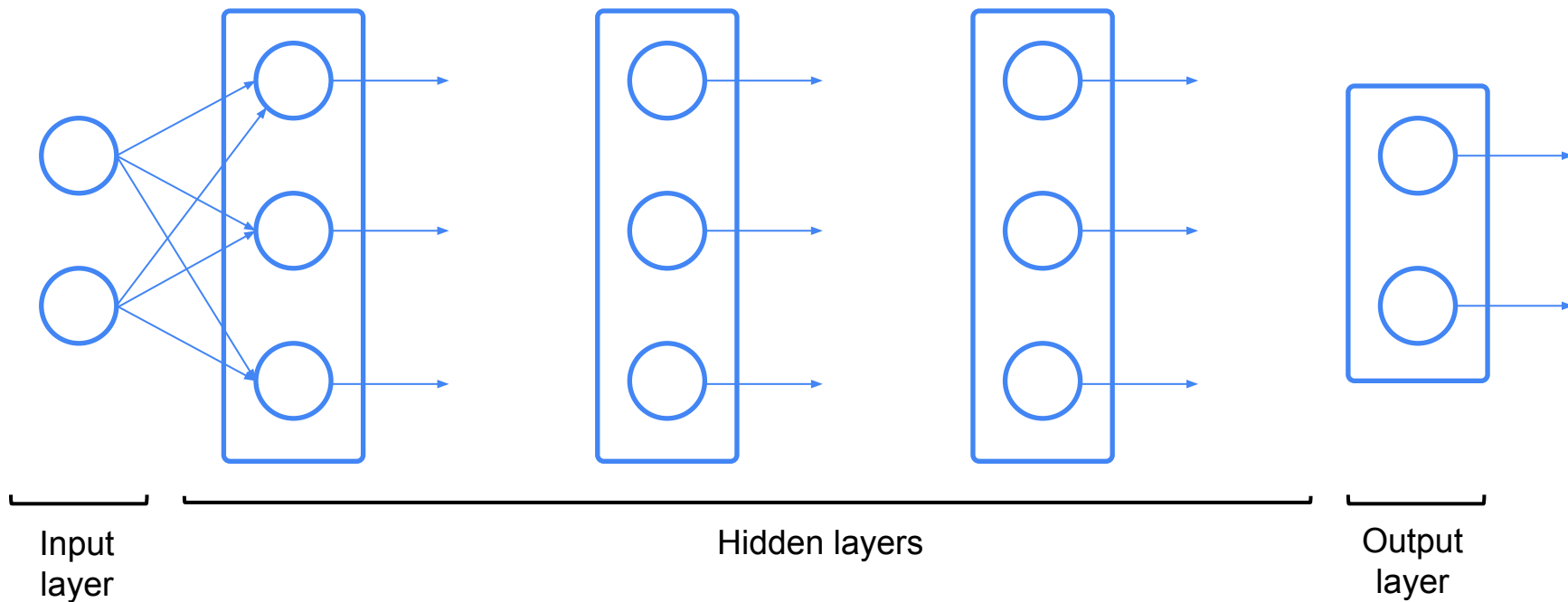
1. **El rol de las funciones de activación**
2. Funciones de Activación lineales
3. Funciones escalonadas (step functions)
4. Funciones Lineales por Partes
(Piecewise) [ReLu]
5. Funciones suaves
6. Softmax
7. ¿Cómo elegir?
8. Ejercicio



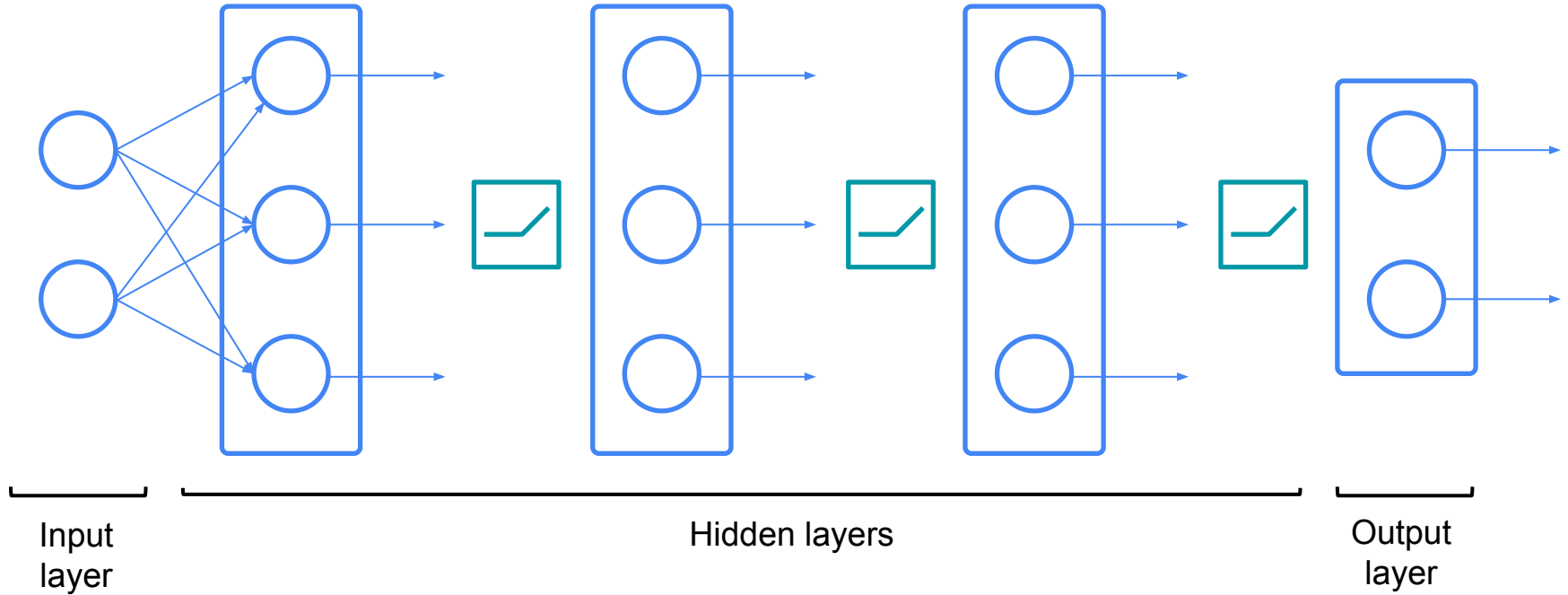
Modelo secuencial



Modelo secuencial



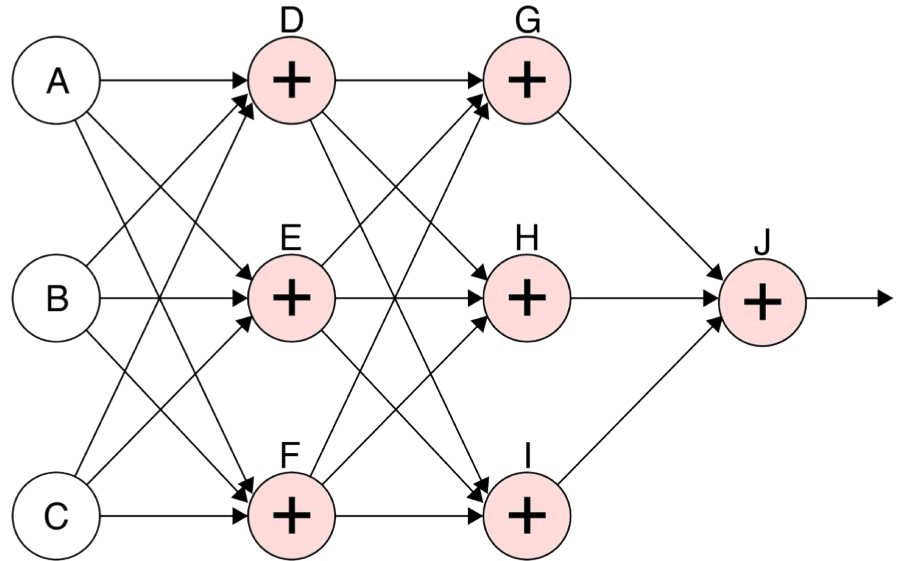
Modelo secuencial



 Función no lineal

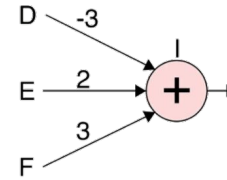
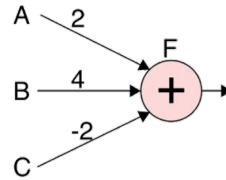
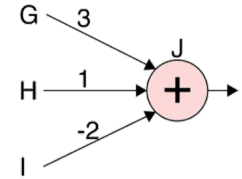
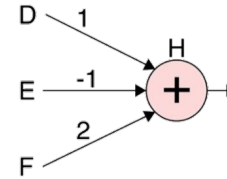
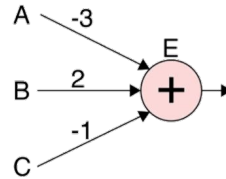
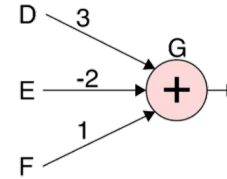
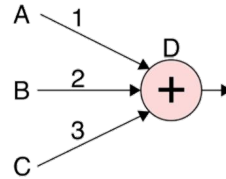
Importancia de las funciones de activación

- Red completamente conectada sin funciones de activación.



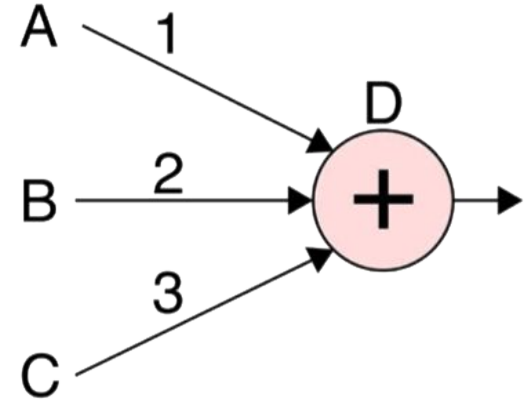
Importancia de las funciones de activación

- Red completamente conectada sin funciones de activación.
- Cada unidad de D a J calcula una combinación de las entradas A, B, y C.



Importancia de las funciones de activación

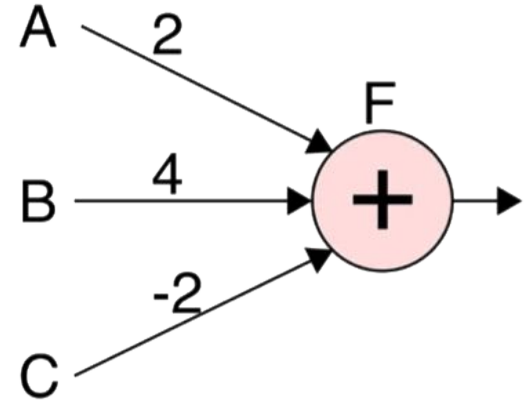
- Red completamente conectada sin funciones de activación.
- Cada unidad de D a J calcula un combinación de las entradas A, B, y C.



$$D = A + 2B + 3C$$

Importancia de las funciones de activación

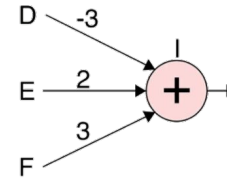
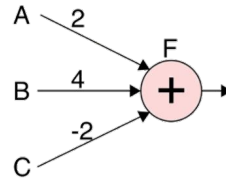
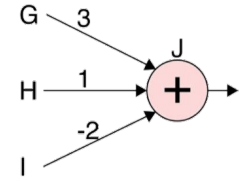
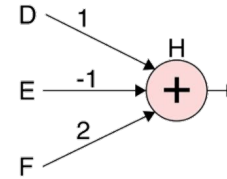
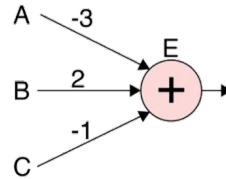
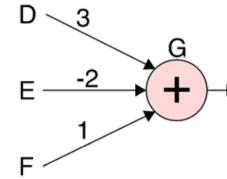
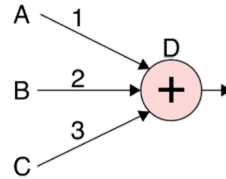
- Red completamente conectada sin funciones de activación.
- Cada unidad de D a J calcula una combinación de las entradas A, B, y C.



$$F = 2A + 4B - 2C$$

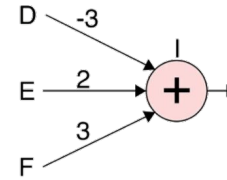
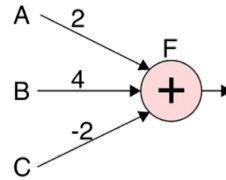
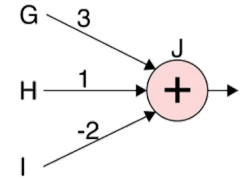
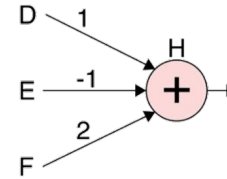
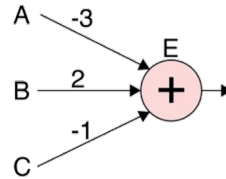
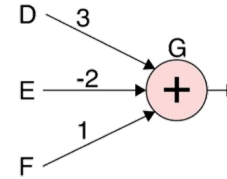
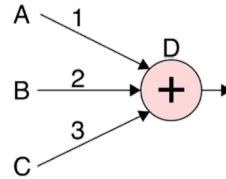
Importancia de las funciones de activación

- Red completamente conectada sin funciones de activación.
- Cada unidad de D a J calcula una combinación de las entradas A, B, y C.



Importancia de las funciones de activación

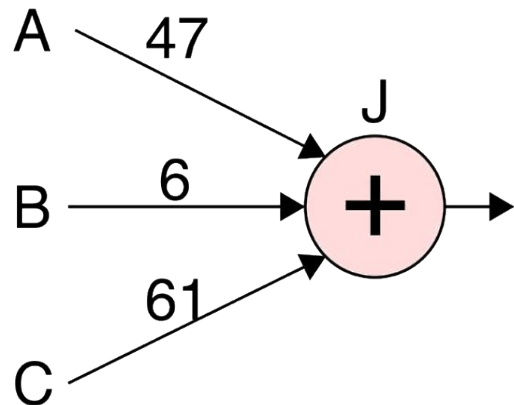
- Red completamente conectada sin funciones de activación.
- Cada unidad de D a J calcula una combinación de las entradas A, B, y C.



$$J = 47A + 6B + 61C$$

Importancia de las funciones de activación

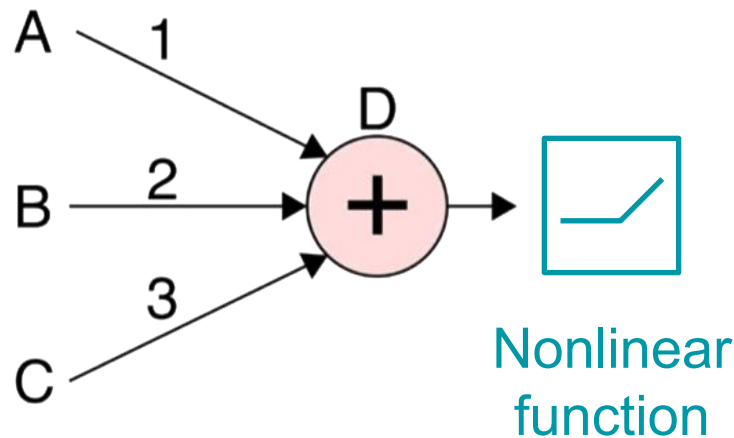
- Red completamente conectada sin funciones de activación.
- Cada unidad de D a J calcula un combinación de las entradas A, B, y C.
- **Colapso de la red**: la salida de J es la combinación de A, B y C.



$$J = 47A + 6B + 61C$$

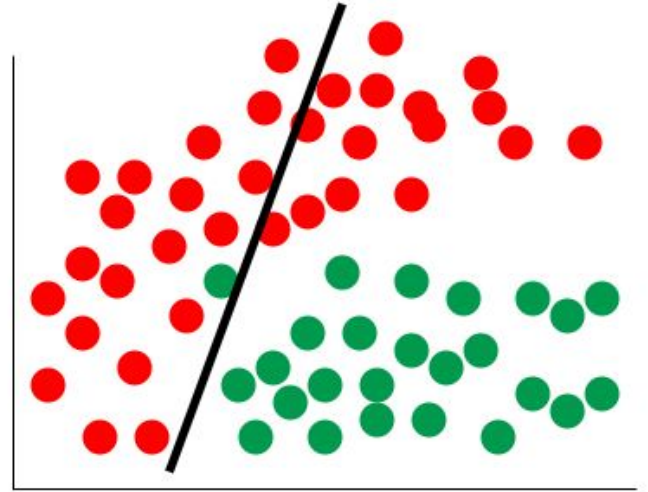
Propósito de las funciones de activación

- **Prevención de colapso:**
función no lineal después de cada unidad.
- Aprender mapeos no lineales
- Ayuda a la red a capturar estructuras complejas de los datos.
- Actúa como puertas de flujo de información a través de la red.



Agenda

1. El rol de las funciones de activación
2. **Funciones de Activación lineales**
3. Funciones escalonadas (step functions)
4. Funciones Lineales por Partes (Piecewise) [ReLU]
5. Funciones suaves
6. Softmax
7. ¿Cómo elegir?
8. Ejercicio

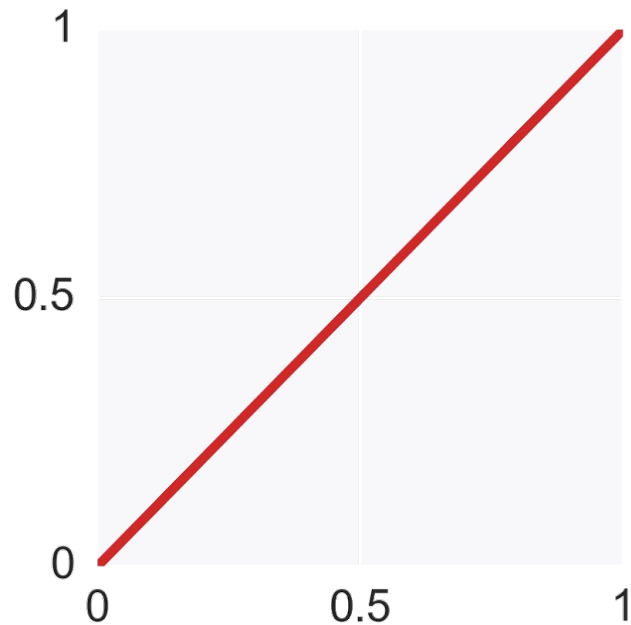


Función de identidad

- Función de identidad:

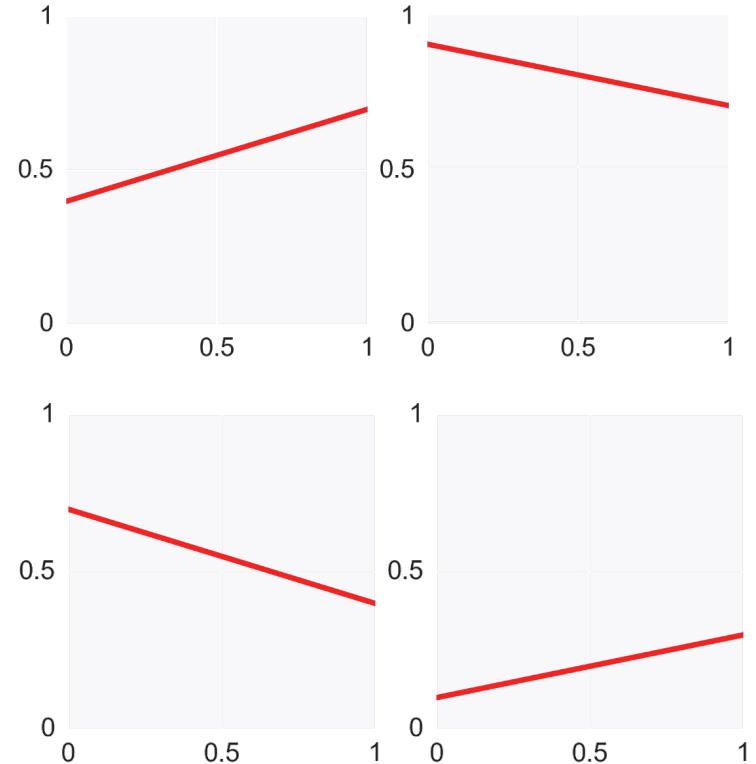
$$f(x) = x$$

- El output es el mismo que la entrada
 - el input pasa por la unidad de la NN sin modificaciones



Variantes

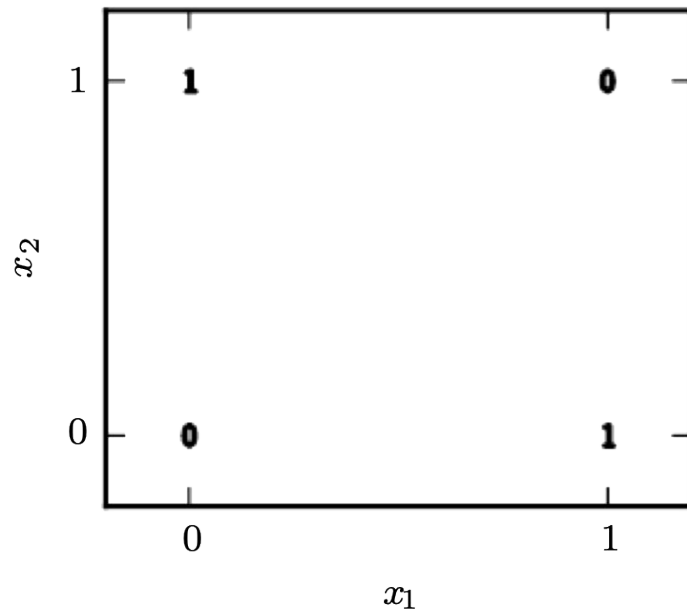
- Variantes de funciones lineales
 - solamente multiplicación y suma
- Las combinaciones lineales del input sufren del **colapso de red**.



Ejemplo: Aprender XOR

- Algunas funciones no se pueden aprender con activaciones lineales
- La función XOR es un operador binario con dos inputs: x_1 y x_2

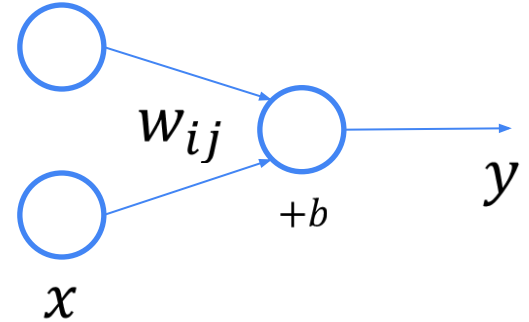
$\left\{ \begin{array}{ll} \circ \text{ output} = \text{True} & \text{si los inputs son diferentes} \\ \circ \text{ output} = \text{False} & \text{sino} \end{array} \right.$



Ejemplo: Aprender XOR

- Algunas funciones no se pueden aprender con activaciones lineales
- La función XOR es un operador binario con dos inputs: x_1 y x_2

$$\left\{ \begin{array}{ll} \circ \text{ output} = \text{True} & (\text{si los inputs son diferentes}) \\ \circ \text{ output} = \text{False} & (\text{sino}) \end{array} \right.$$



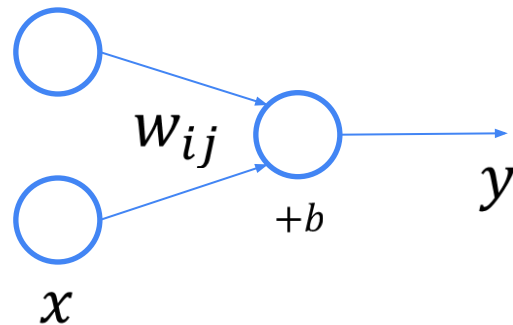
$$y = w_1x_1 + w_2x_2 + b$$

Ejemplo: Aprender XOR

- Algunas funciones no se pueden aprender con activaciones lineales
- La función XOR es un operador binario con dos inputs: x_1 y x_2

$$\left\{ \begin{array}{ll} \circ \text{ output} = \text{True} & (\text{si los inputs son diferentes}) \\ \circ \text{ output} = \text{False} & (\text{sino}) \end{array} \right.$$

- Aprender los pesos para el mapeo:
 - $(0,0) \rightarrow 0$
 - $(0,1) \rightarrow 1$
 - $(1,0) \rightarrow 1$
 - $(1,1) \rightarrow 0$



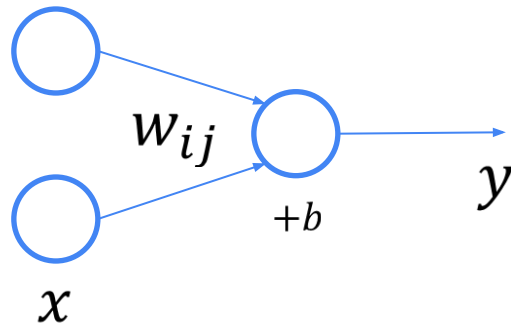
$$y = w_1x_1 + w_2x_2 + b$$

Ejemplo: Aprender XOR

- Algunas funciones no se pueden aprender con activaciones lineales
- La función XOR es un operador binario con dos inputs: x_1 y x_2

$$\left\{ \begin{array}{ll} \circ \text{ output} = \text{True} & (\text{si los inputs son diferentes}) \\ \circ \text{ output} = \text{False} & (\text{sino}) \end{array} \right.$$

- Aprender los pesos para el mapeo:
 - $(0,0) \rightarrow 0$
 - $(0,1) \rightarrow 1$
 - $(1,0) \rightarrow 1$
 - $(1,1) \rightarrow 0$



$$0 = w_1 0 + w_2 0 + b$$

$$1 = w_1 0 + w_2 1 + b$$

$$1 = w_1 1 + w_2 0 + b$$

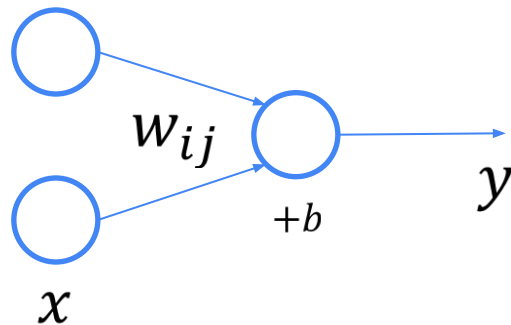
$$0 = w_1 1 + w_2 1 + b$$

Ejemplo: Aprender XOR

- Algunas funciones no se pueden aprender con activaciones lineales
- La función XOR es un operador binario con dos inputs: x_1 y x_2

$$\left\{ \begin{array}{ll} \circ \text{ output} = \text{True} & (\text{si los inputs son diferentes}) \\ \circ \text{ output} = \text{False} & (\text{sino}) \end{array} \right.$$

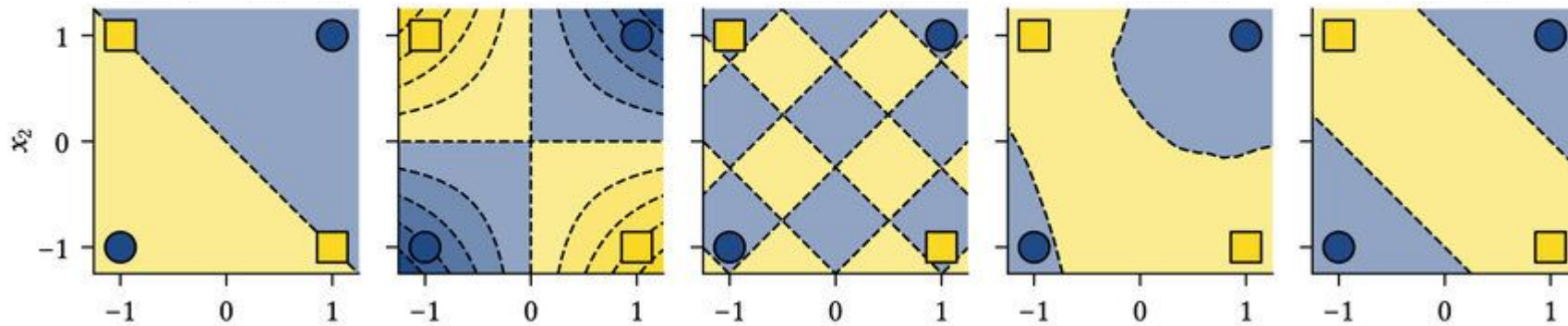
- Aprender los pesos para el mapeo:
 - $(0,0) \rightarrow 0$
 - $(0,1) \rightarrow 1$
 - $(1,0) \rightarrow 1$
 - $(1,1) \rightarrow 0$



$$\begin{aligned} 0 &= b \\ 1 &= w_2 \\ 1 &= w_1 \\ 0 &= w_1 + w_2 \end{aligned}$$

Ejemplo: Aprender XOR

- El modelo lineal no puede describir la función XOR
 - no es linealmente separable
- Es necesario otras funciones de activación que describen comportamientos más complejos.



Pros y Cons de Activaciones Lineales

● Ventajas

- Simples y computacionalmente eficientes
- Fáciles de interpretar



● Desventajas

- Limitadas frente a patrones de datos más complejos
- Capas apiladas con activaciones lineales van a producir un colapso de red
→ transformaciones lineales.
- Generalmente se utilizan solamente en la neuronas de salida



Agenda

1. El rol de las funciones de activación
2. Funciones de Activación lineales
3. **Funciones escalonadas (step functions)**
4. Funciones Lineales por Partes (Piecewise) [ReLU]
5. Funciones suaves
6. Softmax
7. ¿Cómo elegir?
8. Ejercicio



Funciones escalonadas

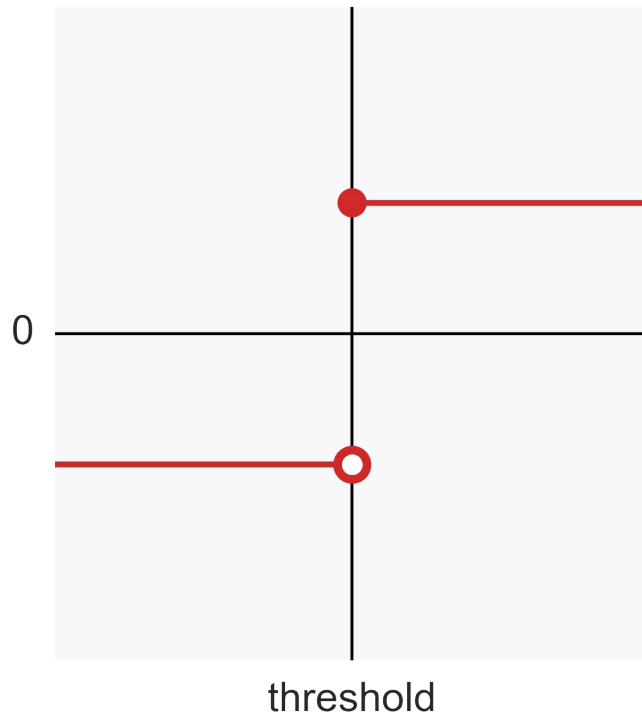
- La función lineal más simple

- Identidad

- es igual a no tener ninguna función de activación.

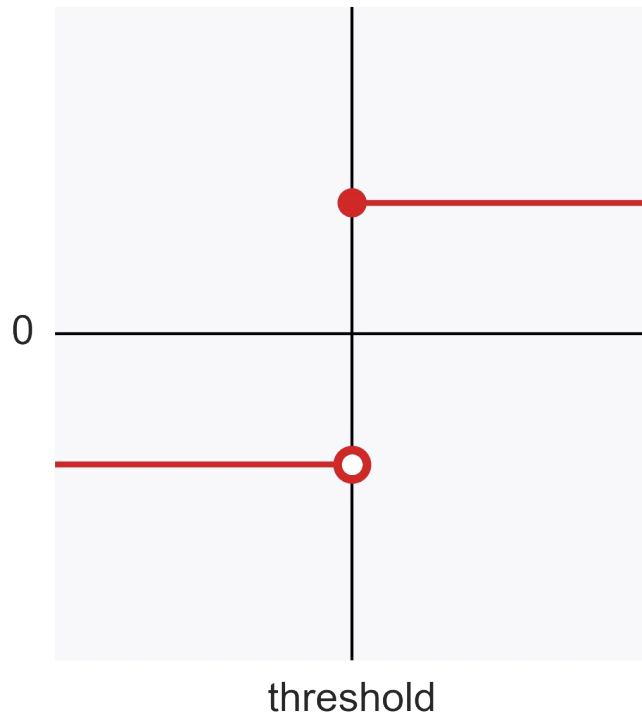
- La función no-lineal más simple

- la escalonada.



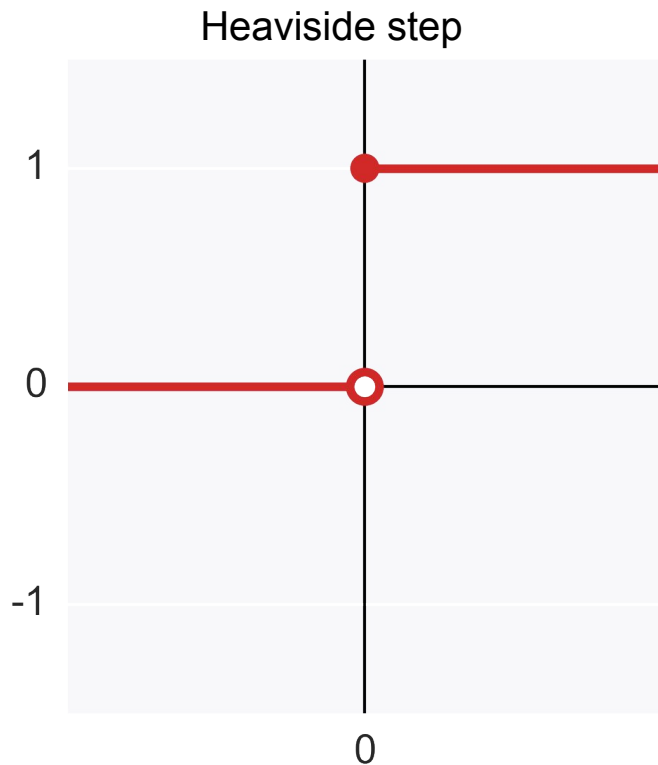
Funciones escalonadas

- Tiene **dos valores fijos** uno para la izquierda y otro para la derecha de un valor umbral (**threshold**).



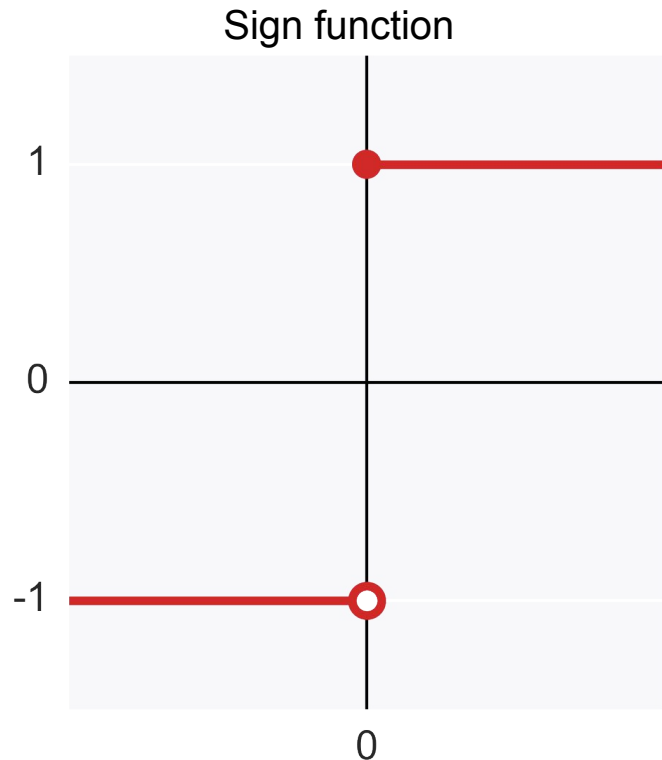
Heaviside Step

- La función escalonada tiene **dos valores fijos**, uno para la izquierda y otro para la derecha de un valor umbral (**threshold**).
- Cuando el valor izquierdo es 0, se llama **Heaviside step** (lado pesado?).



Función de signo

- La función escalonada tiene **dos valores fijos**, uno para la izquierda y otro para la derecha de un valor umbral (**threshold**).
- Una función Heaviside step con valor -1 a la izquierda es llamada la función de signo (**sign function**).



Pros y Cons de la funciones escalonadas

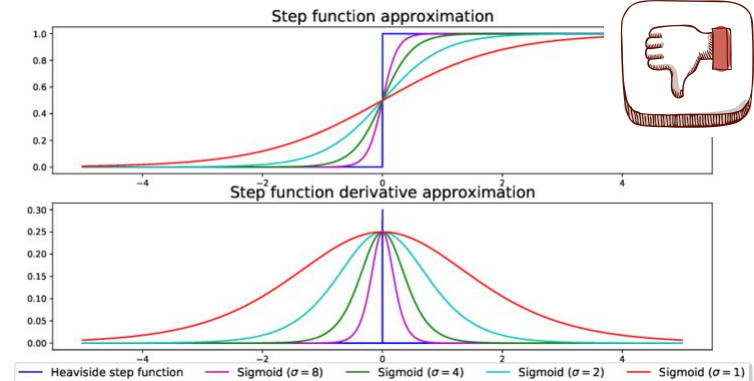
● Ventajas

- Simples y fáciles de implementar
- Salida binaria sirve para algunas tareas (Ej. clasificación).
- Históricamente importante (Ej. perceptron)



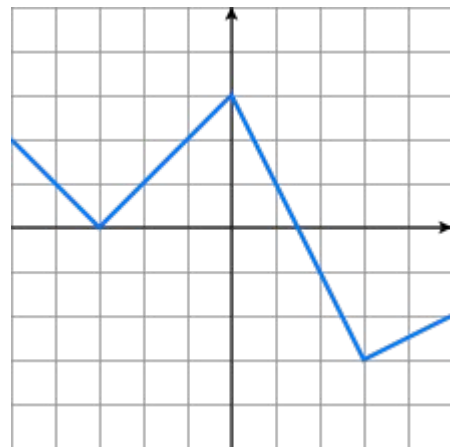
● Desventajas

- No diferenciable en el threshold
- Sensible a pequeños cambios en la entrada.
- No tiene información de gradiente para aprender.



Agenda

1. El rol de las funciones de activación
2. Funciones de Activación lineales
3. Funciones escalonadas (step functions)
4. **Funciones Lineales por Partes (Piecewise) [ReLu]**
5. Funciones suaves
6. Softmax
7. ¿Cómo elegir?
8. Ejercicio

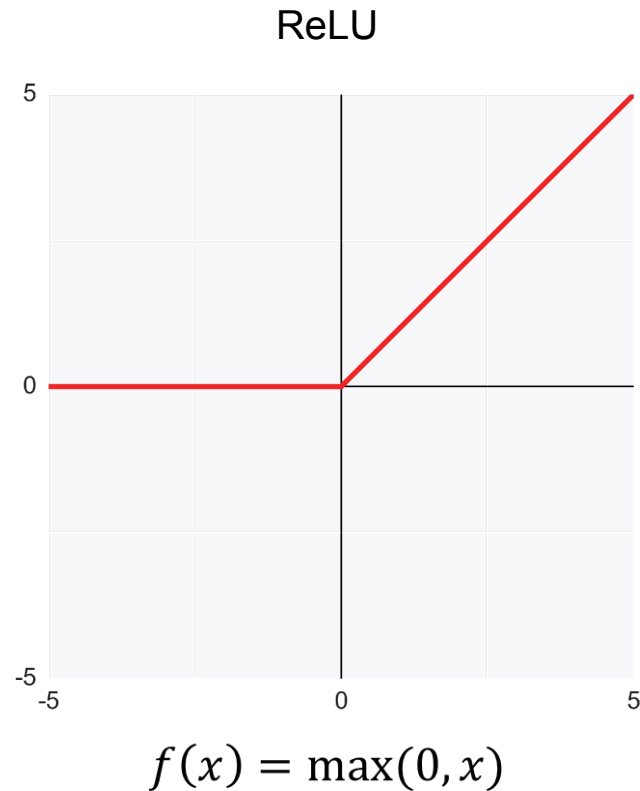


Funciones lineales por partes

- Son funciones de varias “partes” y cada una de ellas es una línea.

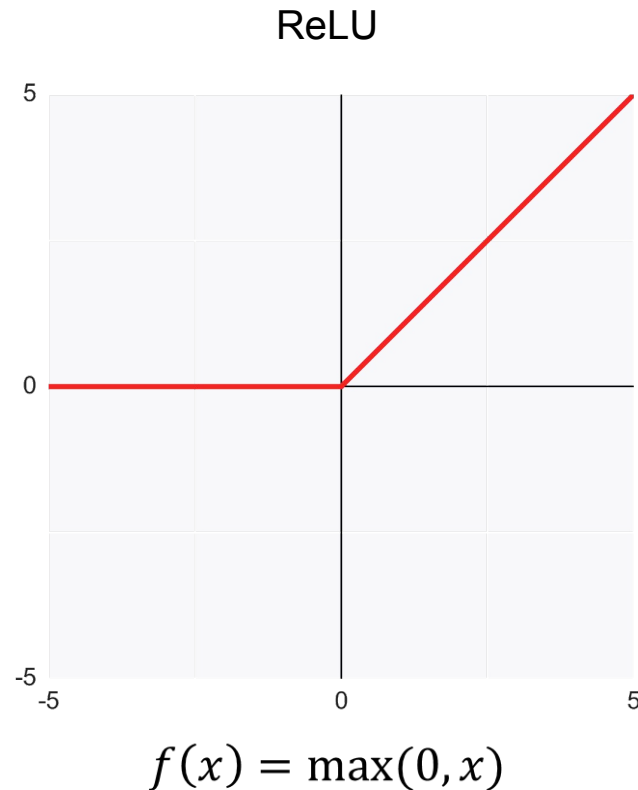
The Rectified Linear Unit (ReLU)

- Son funciones de varias “partes” y cada una de ellas es una línea.
- La más popular es la **rectified linear unit (ReLU)**



The Rectified Linear Unit (ReLU)

- Son funciones de varias “partes” y cada una de ellas es una línea.
- La más popular es la **rectified linear unit (ReLU)**
- Si el input es negativo el output es cero. Sino, es la identidad.

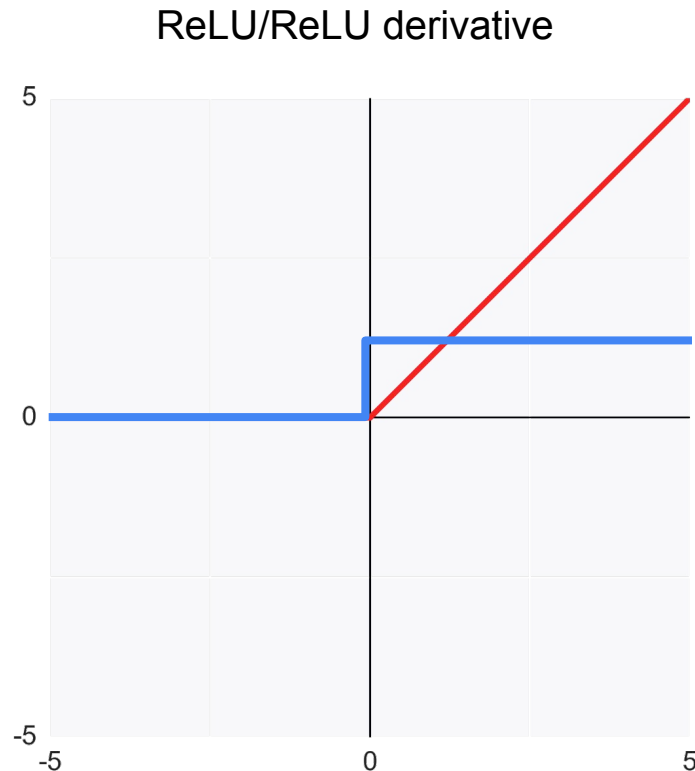


El problema del **Vanishing Gradient**

- Los gradientes se hacen muy pequeños durante backpropagation a través de las capas

El problema del Vanishing Gradient

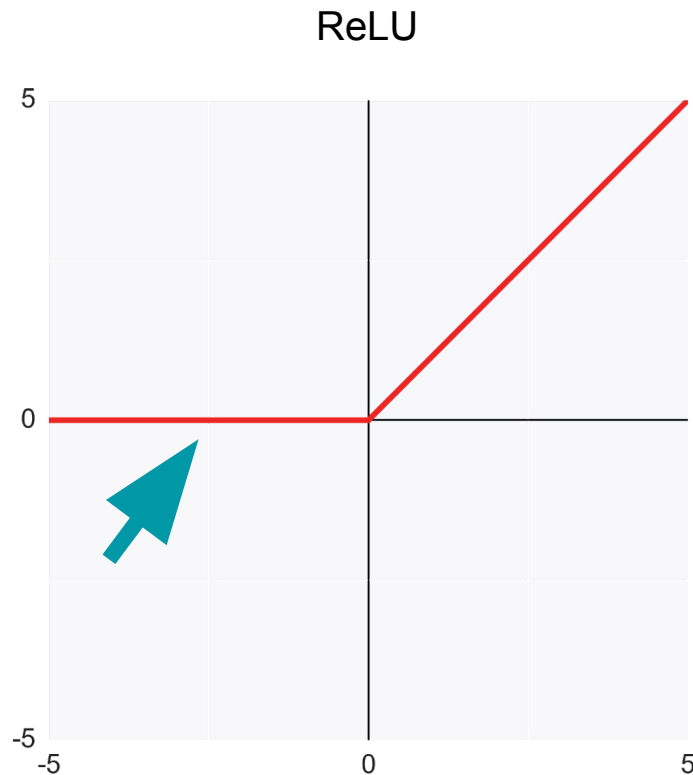
- Los gradientes se hacen muy pequeños durante backpropagation a través de las capas
- La derivada de la ReLU es 0 o 1 dependiendo del valor de entrada, aliviando el problema del vanishing gradient.



The Dying ReLU Problem

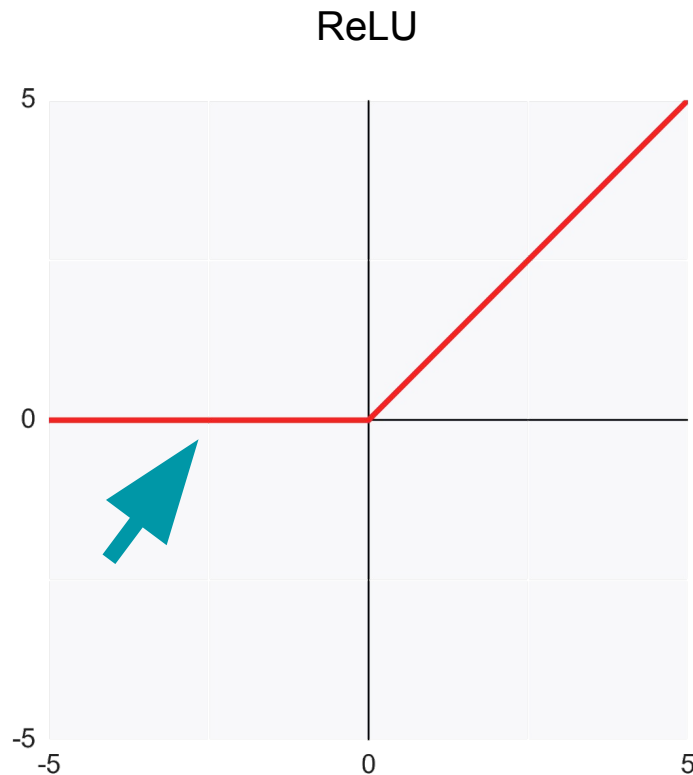
- La función de ReLU puede llevar a neuronas muertas

→ neuronas que siempre están inactivas.



The Dying ReLU Problem

- La función de ReLU puede llevar a neuronas muertas
 - neuronas que siempre están inactivas.
- Durante backpropagation el gradiente de estas neuronas es cero
 - **no se actualizan sus pesos.**



ReLU Pros y Cons

● Ventajas

- Simple y computacionalmente eficiente
- Introduce **no linealidad en la red**, lo que permite aprender patrones complejos.
- Ayuda con el problema del **vanishing gradient**, que puede ocurrir en redes neuronales profundas con funciones de activación tradicionales.



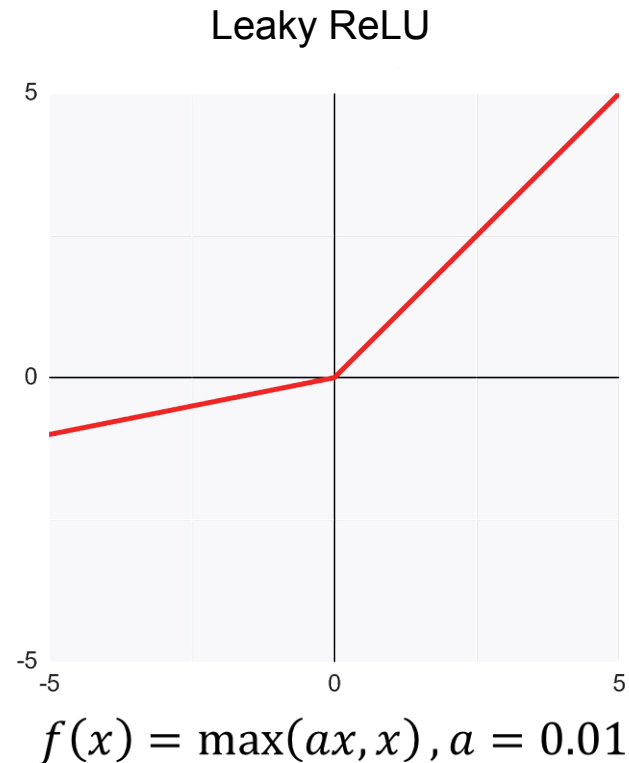
● Desventajas

- Neuronas con input negativo pueden llegar a desactivarse
→ disminución de capacidad de aprendizaje de la red



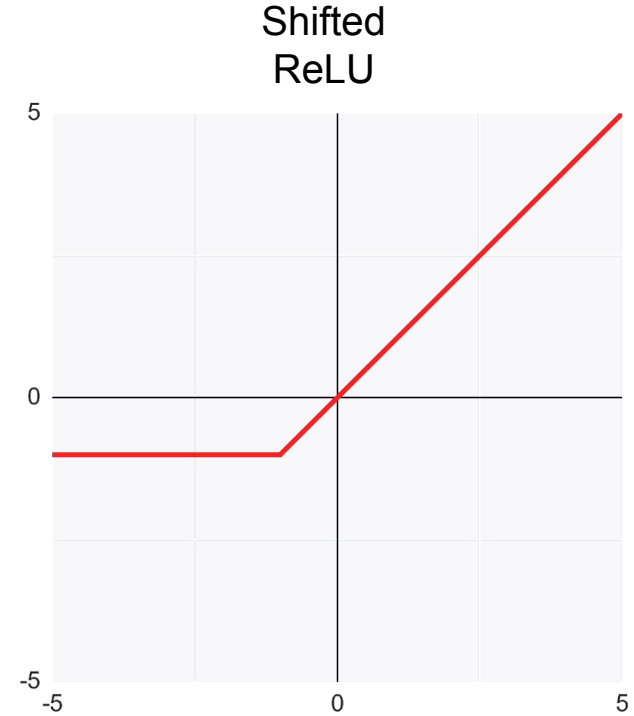
The Leaky ReLU

- Cambia el resultado para los valores negativos.
- Como output es el input achicado por un factor (Ej, 10).
 - Si el factor es 1, tenemos una linea recta y podemos sufrir el colapso de la neurona.



The Shifted ReLU

- Corre el punto de “cambio” para atrás.
 - permite a la función tener outputs distintos a cero para algunos inputs negativos.



Variantes ReLU: Pros and Cons

● Ventajas

- Permiten el threshold de activación variable, lo cual se puede ajustar mejor en ciertos dominios.
- Evita el problema de la neurona muerta (dying ReLU)
- Mantiene los beneficios de la ReLU, no linealidad y eficiencia computacional.



● Desventajas

- Introduce hyper parámetros adicionales:
 - parámetro de shift requiere tuneo



Maxout Networks

Ian J. Goodfellow
David Warde-Farley
Mehdi Mirza
Aaron Courville
Yoshua Bengio

GOODFELI@IRO.UMONTREAL.CA
WARDEFAR@IRO.UMONTREAL.CA
MIRZAMOM@IRO.UMONTREAL.CA
AARON.COURVILLE@UMONTREAL.CA
YOSHUA.BENGIO@UMONTREAL.CA

Département d'Informatique et de Recherche Opérationnelle, Université de Montréal
2920, chemin de la Tour, Montréal, Québec, Canada, H3T 1J8

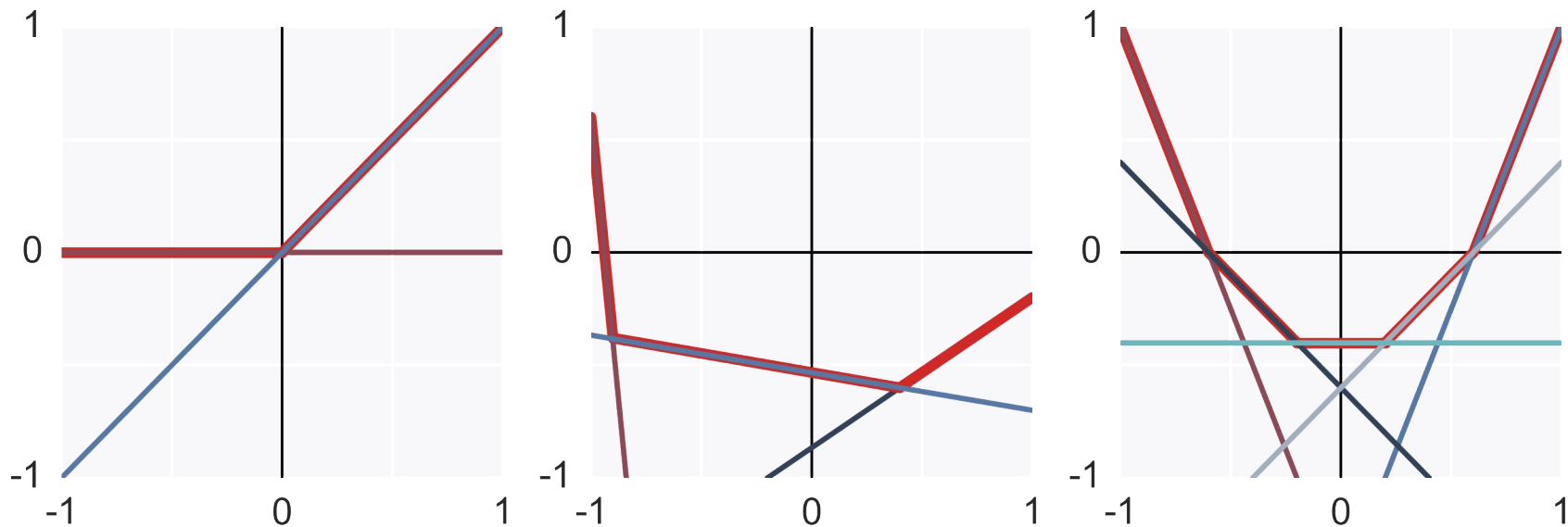
Abstract

We consider the problem of designing models to leverage a recently introduced approximate model averaging technique called *dropout*. We define a simple new model called *maxout* (so named because its *output* is the *max* of a set of inputs, and because it is a natural companion to dropout) designed to both facilitate optimization by dropout and im

Dropout is generally viewed as an indiscriminately applicable tool that reliably yields a modest improvement in performance when applied to almost any model.

We argue that rather than using dropout as a slight performance enhancement applied to arbitrary models, the best performance may be obtained by directly designing a model that enhances dropout's abilities as a model averaging technique. Training using dropout differs significantly from previous approaches such as

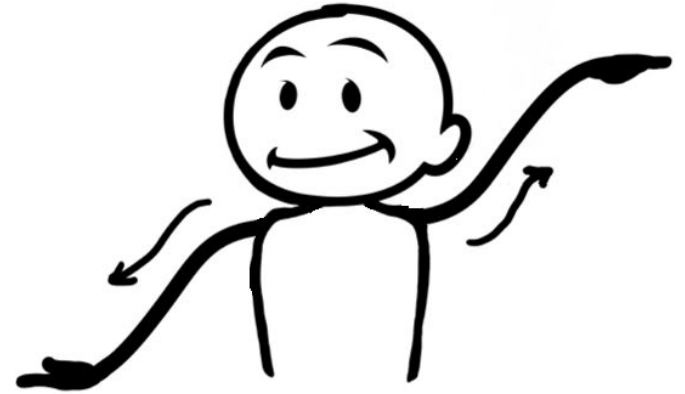
Maxout Activation Functions



Evita dying neurons

Agenda

1. El rol de las funciones de activación
2. Funciones de Activación lineales
3. Funciones escalonadas (step functions)
4. Funciones Lineales por Partes (Piecewise) [ReLU]
5. **Funciones suaves**
6. Softmax
7. ¿Cómo elegir?
8. Ejercicio



$$y = \frac{x}{(1 + |x|)}$$

Funciones suaves

- Las funciones por partes lineales son:

- Fáciles de implementar
- Computacionalmente eficientes
- No diferenciables en el punto de inflexión (arbitrariamente elegido)

- Las funciones de activación suaves son:

- continuas
- completamente diferenciables

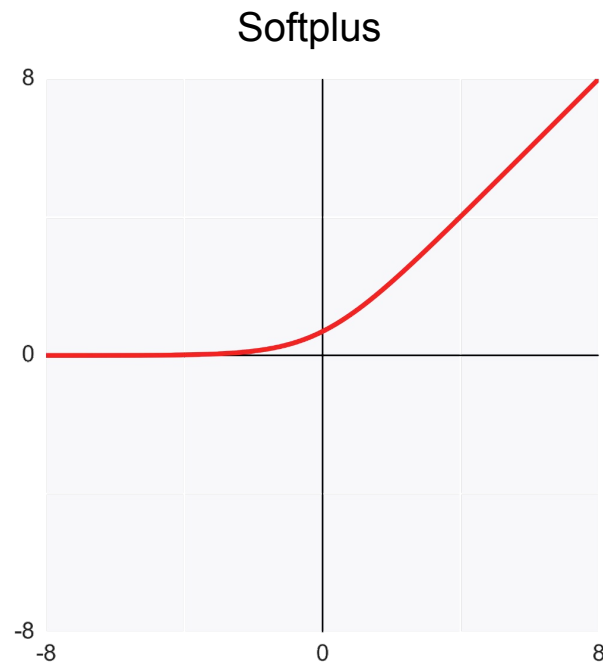
→ **Gradientes continuos resultan, en general, en optimizaciones más estables.**

Motivación a funciones suaves

- **Diferenciables en todo el dominio:** son útiles para los algoritmos de optimización que a veces necesitan métodos de segundo orden (Hessiano).
- **Convergencia:** pueden converger mejor por los gradientes continuos y más estables, pero pueden ser más lentos → vanishing gradient
- Útil para redes neuronales recurrentes y long short term memory (LSTM) que necesitan mantener “vivos” a los estados ocultos.

Función Softplus

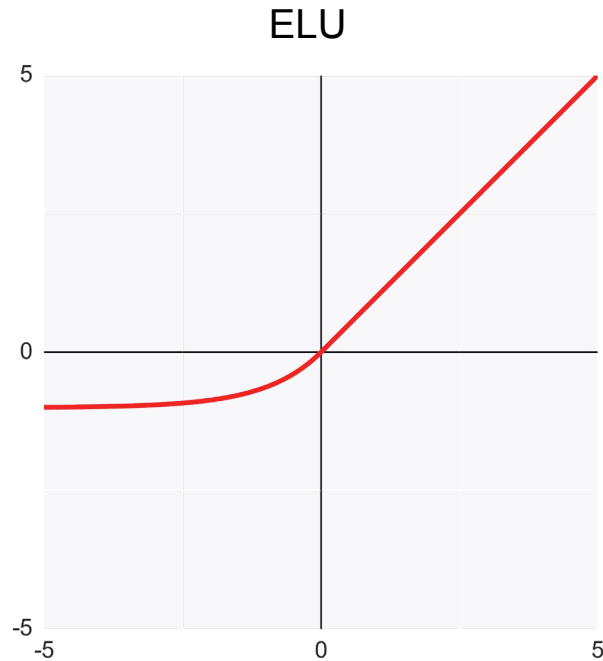
- Versión suave de la ReLU
 - Continua
 - Diferenciable
- Más costosa computacionalmente
 - Exponentes
 - Logaritmos



$$f(x) = \ln(1 + e^x)$$

The Exponential Linear Unit (ELU)

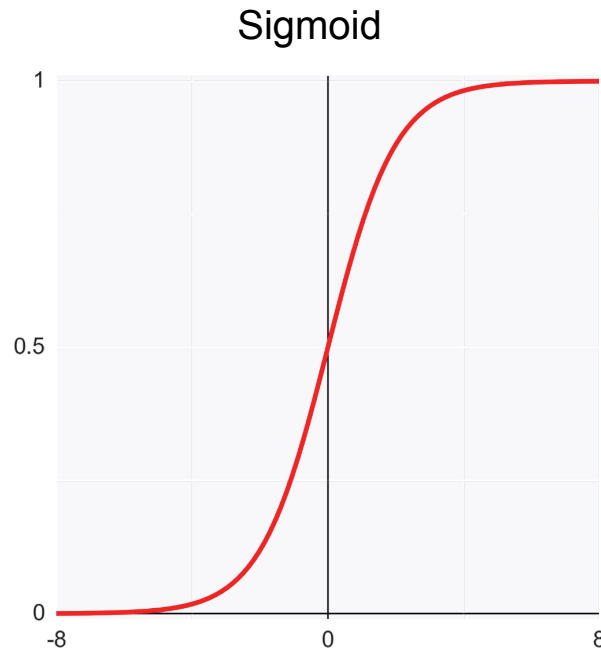
- Versión suave de la **shifted ReLU**
- Permite pequeños, gradientes distintos de cero para inputs negativos
- El parámetro α es un escalar positivo, típicamente seteado en 1



$$f(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

The Sigmoid/Logistic Function

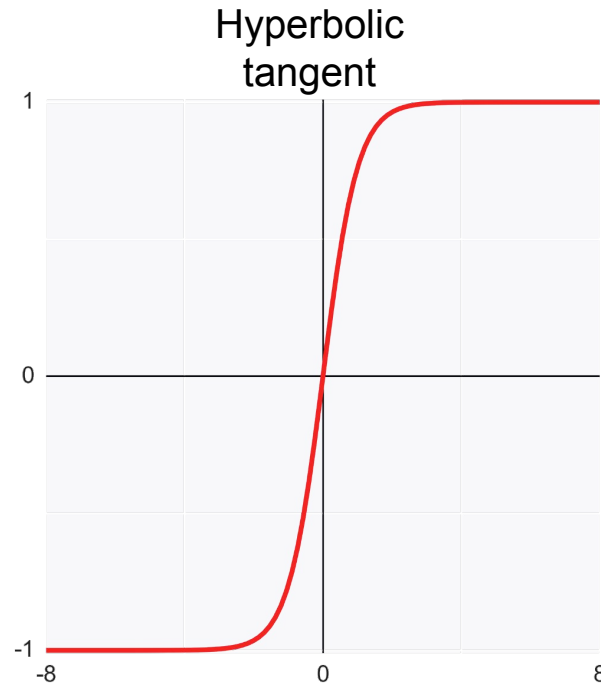
- Curva con forma de S
 - muy usada en NN
- Su output es entre 0 y 1
 - sirve para clasificación
- Gradientes son muy chicos para valores lejos de cero
 - **vanishing gradient** problem



$$f(x) = \frac{e^x}{1 + e^x}$$

The Hyperbolic Tangent (Tanh)

- Suave curva con forma de S
- El output varía entre -1 y 1
 - centrado en 0
- Computacionalmente costosa
- Sufre del problema de **vanishing gradient**



$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

SWISH: A SELF-GATED ACTIVATION FUNCTION

Prajit Ramachandran*, Barret Zoph, Quoc V. Le

Google Brain

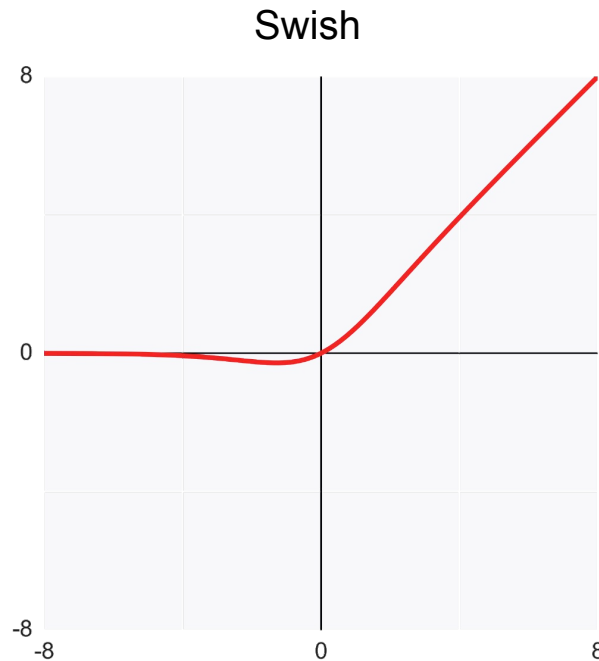
{prajit, barretzoph, qvl}@google.com

ABSTRACT

The choice of activation functions in deep networks has a significant effect on the training dynamics and task performance. Currently, the most successful and widely-used activation function is the Rectified Linear Unit (ReLU). Although various alternatives to ReLU have been proposed, none have managed to replace it due to inconsistent gains. In this work, we propose a new activation function, named Swish, which is simply $f(x) = x \cdot \text{sigmoid}(x)$. Our experiments show that Swish tends to work better than ReLU on deeper models across a number of challenging datasets. For example, simply replacing ReLUs with Swish units improves top-1 classification accuracy on ImageNet by 0.9% for Mobile NASNet-A and 0.6% for Inception-ResNet-v2. The simplicity of Swish and its similarity to ReLU make it easy for practitioners to replace ReLUs with Swish units in any neural network.

The Swish (or SiLU) Function

- Introducido por investigadores en Google Brain
- Combinación entre la ReLU básica y la forma sigmoid
- El parámetro β puede aprenderse o ser fijado con un escalar, en general 1.
- Mecanismo de **self-gating**



$$f(x) = \frac{x}{1 + e^{-\beta x}}$$

Pros

- Suaves y diferenciables

- Sirven para varios algoritmos de optimización
- Mejores propiedades de convergencia en algunos casos

- Ventajas específicas

- Sigmoid: output $(0, 1)$ útil para clasificación
- tanh: output $(-1, 1)$ centrado en 0, mejor para convergencia
- Swish: mecanismo de self-gating para adaptaciones
- ELU: ataca el problema de la neurona muerta en ReLU (dying neuron)



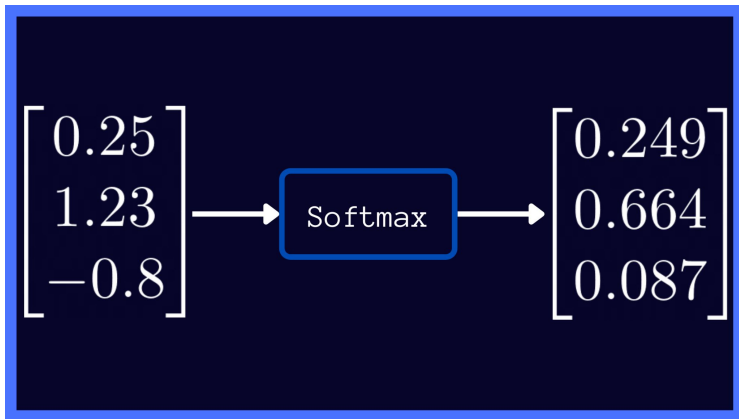
Cons

- Costo computacional
 - Más costosas en general que las lineales por partes
 - exponentes y multiplicaciones
- Saturación y vanishing gradients
 - Sigmoid y tanh pueden sufrir del vanishing gradients
 - Swish y ELU mitigan el problema (un poco)



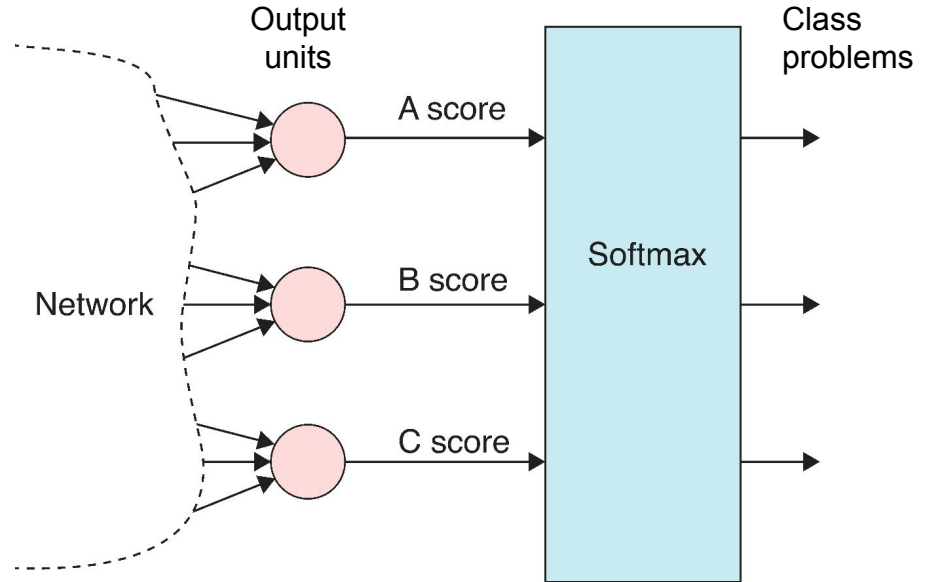
Agenda

1. El rol de las funciones de activación
2. Funciones de Activación lineales
3. Funciones escalonadas (step functions)
4. Funciones Lineales por Partes (Piecewise) [ReLU]
5. Funciones suaves
6. **Softmax**
7. ¿Cómo elegir?
8. Ejercicio



Función Softmax

- Operación especial
- Típicamente aplicada solamente en la **neurona output de un clasificador**
- No es estrictamente una función de activación:
 - Se aplica simultáneamente a todas las neuronas de salida



Función Softmax

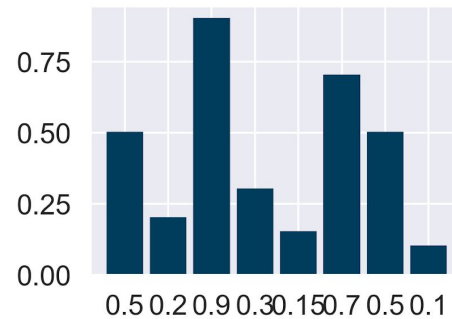
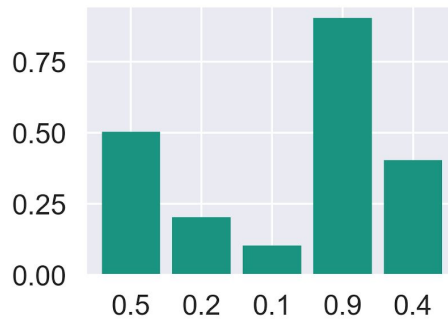
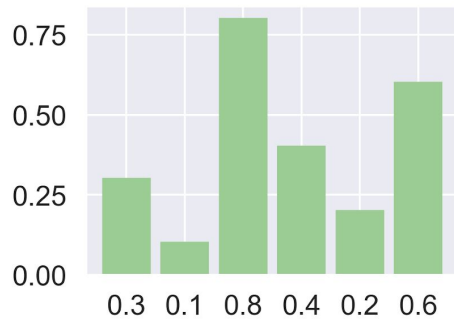
- Toma un vector de entrada x de K números reales y los normaliza a una distribución de probabilidad, consistiendo en K probabilidades.

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}}$$

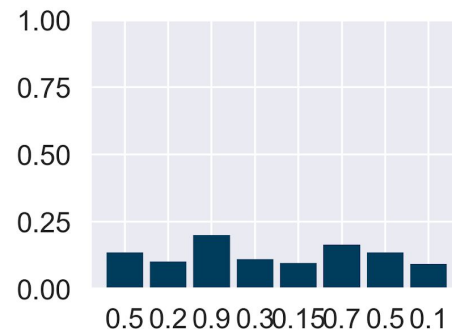
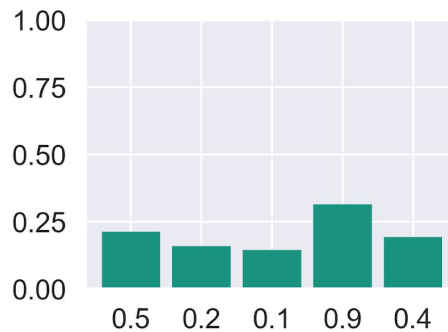
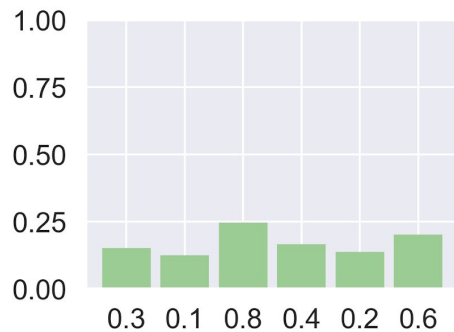
- La función exponencial normalizada asegura que la suma de las probabilidades entre todas las clases es igual a 1

La función softmax

Predicted
scores



Scores after
softmax



Log Softmax

- En lugar de devolver las probabilidades directas, devuelve el logaritmo de las probabilidades.
- Numéricamente estable, especialmente cuando los valores en el vector de entrada son grandes o cuando se realizan cálculos repetidos.

$$\text{LogSoftmax}(x_i) = \log \left(\frac{\exp(x_i)}{\sum_j \exp(x_j)} \right)$$

Agenda

1. El rol de las funciones de activación
2. Funciones de Activación lineales
3. Funciones escalonadas (step functions)
4. Funciones Lineales por Partes
(Piecewise) [ReLu]
5. Funciones suaves
6. Softmax
7. **¿Cómo elegir?**
8. Ejercicio

¿Cómo elegir?

- Dominio del problema y sus características
- Profundidad de la red y arquitectura
- Balance entre complejidad y performance
- Experimentación y validación...

¿Cómo elegir?

- Funciones de activación custom
- Modelos pre-entrenados y transfer learning (vienen ya seleccionados)
- Mantenerse actualizado con la última investigación
- Aprender continuamente y mejorar

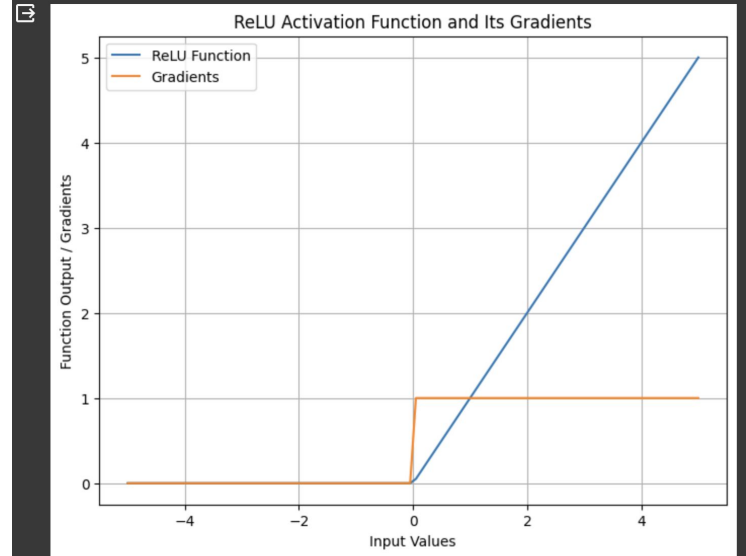
Agenda

1. El rol de las funciones de activación
2. Funciones de Activación lineales
3. Funciones escalonadas (step functions)
4. Funciones Lineales por Partes
(Piecewise) [ReLu]
5. Funciones suaves
6. Softmax
7. ¿Cómo elegir?
8. **Ejercicio**

Activaciones, derivadas y MNIST.ipynb

1. Implementar distintas funciones y plotear sus gradientes:
<https://pytorch.org/docs/stable/nn.html#non-linear-activations-weighted-sum-nonlinearity>
2. Evaluar el clasificador multicapa de MNIST con las distintas funciones y comparar sus loss.

```
# Plot the input values, ReLU function, and their gradients
plt.figure(figsize=(8, 6))
plt.plot(x.detach().numpy(), y.detach().numpy(), label='ReLU Function')
plt.plot(x.detach().numpy(), gradients.detach().numpy(), label='Gradients')
plt.title("ReLU Activation Function and Its Gradients")
plt.xlabel("Input Values")
plt.ylabel("Function Output / Gradients")
plt.grid()
plt.legend()
plt.show()
```



Bibliografía

- [Deep Learning](#), Ian Goodfellow and Yoshua Bengio and Aaron Courville - Capítulo 6.5
- Neural Networks and Deep Learning. Charu C. Aggarwal 1.3
- SWISH: A SELF-GATED ACTIVATION FUNCTION Prajit Ramachandran* , Barret Zoph, Quoc V. Le Google Brain {prajit,barretzoph,qvl}@google.com
- Maxout Networks Ian J. Goodfellow, David Warde-Farley , Mehdi Mirza , Aaron Courville, Yoshua Bengio